

Managing Application Configuration and Secrets

Intermediate Kubernetes — Module 5 of 13

Enterprise EKS Platform





Module Agenda

- The Twelve-Factor App — Config Principle
- ConfigMaps: Creation and Consumption Patterns
- Secrets: Types, Creation, and Consumption
- Environment Variable Precedence
- Volume Projection and subPath Mounts
- Immutable ConfigMaps/Secrets and Encryption at Rest
- External Secrets Operator with Vault



The Twelve-Factor App

Factor III: Store Config in the Environment



Why Externalize Configuration?

Anti-Pattern: Embedded Config

- Config hardcoded in application code
- Requires rebuilding images for each environment
- Secrets committed to source control
- No separation of concerns

Best Practice: External Config

- Config injected at runtime
- Same image across dev, staging, prod
- Secrets managed independently
- Config changes without redeployment

Kubernetes Config Primitives

Kubernetes provides two native resources for external configuration:

- **ConfigMaps** — Non-sensitive configuration data (key-value pairs, files)
- **Secrets** — Sensitive data (credentials, tokens, certificates)



ConfigMaps

Externalizing Non-Sensitive Configuration

Creating ConfigMaps — From Literals

Use `--from-literal` for individual key-value pairs:

```
# Create ConfigMap from literal values
kubectl create configmap app-config \
  --from-literal=DATABASE_HOST=postgres.db.svc.cluster.local \
  --from-literal=DATABASE_PORT=5432 \
  --from-literal=LOG_LEVEL=info \
  --from-literal=CACHE_TTL=300
```

```
# Verify the ConfigMap
kubectl get configmap app-config -o yaml
```



Creating ConfigMaps — From Files

Load entire configuration files into a ConfigMap:

```
# Create from a single file (key = filename)
kubectl create configmap nginx-conf \
  --from-file=nginx.conf

# Create from a file with a custom key name
kubectl create configmap nginx-conf \
  --from-file=main-config=nginx.conf

# Create from a directory (each file becomes a key)
kubectl create configmap app-configs \
  --from-file=./config-dir/
```

Key Insight: When using `--from-file`, the filename becomes the key and the file contents become the value. With a directory, each file in the directory becomes a separate key.



Creating ConfigMaps — Declarative YAML

The recommended approach for managed workloads:

```
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config, namespace: mycompany-apps }
data:
  DATABASE_HOST: "postgres.db.svc.cluster.local"
  LOG_LEVEL: "info"
  application.yaml: |    # multi-line file content
    server:
      port: 8080
  # ...
```

Consuming ConfigMaps as Environment Variables

Individual Keys with valueFrom

```
spec:
  containers:
    - name: app
      image: mycompany/dns-service:1.4
      env:
        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: DATABASE_HOST
        - name: DB_PORT
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: DATABASE_PORT
```

All Keys with envFrom

```
spec:
  containers:
    - name: app
      image: mycompany/dns-service:1.4
      envFrom:
        - configMapRef:
            name: app-config
            prefix: CFG_ # optional
```



Consuming ConfigMaps as Volumes

```
spec:
  containers:
  - name: app
    image: mycompany/dns-service:1.4
    volumeMounts:
    - { name: config-volume, mountPath: /etc/app/config, readOnly: true }
  volumes:
  - name: config-volume
    configMap:
      name: app-config
      items:
      - { key: application.yaml, path: application.yaml }
      - { key: LOG_LEVEL, path: log-level.txt }
```



Hot Reload Behavior Deep Dive

How Volume-Mounted ConfigMap Updates Work

- Kubelet periodically syncs ConfigMap data (default: every 60s)
- Updates use atomic symlink swap: `..data` → `..data_tmp` → rename
- All files update atomically together
- Total propagation delay: kubelet sync + API server cache TTL

Kubernetes Secrets

Managing Sensitive Configuration Data

Secret Types in Kubernetes

Type	Description	Use Case
Opaque	Generic, arbitrary user data	Passwords, API keys, tokens
<code>kubernetes.io/tls</code>	TLS cert + private key	Ingress TLS termination
<code>kubernetes.io/dockerconfigjson</code>	Docker registry credentials	Pulling from private registries
<code>kubernetes.io/basic-auth</code>	Basic auth credentials	Username/password pairs
<code>kubernetes.io/ssh-auth</code>	SSH private key	Git clone, SSH access
<code>kubernetes.io/service-account-token</code>	Service account token	API authentication (legacy)



Creating Secrets

```
# Opaque secret from literals
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password='S3cur3P@ssw0rd!'

# TLS secret from certificate files
kubectl create secret tls app-tls-cert \
  --cert=tls.crt \
  --key=tls.key

# Docker registry secret
kubectl create secret docker-registry ecr-creds \
  --docker-server=123456789.dkr.ecr.us-east-2.amazonaws.com \
  --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password)
```



Declarative Secret YAML

```
# Option A: base64-encoded values
data:
  username: YWRtaW4=
  password: UzNjdXIzUEBzc3cwcmQh

# Option B: plain-text (converted to base64 on storage)
stringData:
  username: admin
  password: "S3cur3P@ssw0rd!"
```




Consuming Secrets in Pods

```
spec:
  containers:
  - name: app
    image: mycompany/dns-service:1.4
    env:
      - name: DB_USERNAME
        valueFrom: { secretKeyRef: { name: db-credentials, key: username } }
      - name: DB_PASSWORD
        valueFrom: { secretKeyRef: { name: db-credentials, key: password } }
      # Mount secrets as files
    volumeMounts:
      - { name: tls-certs, mountPath: /etc/tls, readOnly: true }
  volumes:
  - name: tls-certs
    secret: { secretName: app-tls-cert, defaultMode: 0400 }
```



Environment Variable Precedence

Understanding Override Order



Environment Variable Precedence Rules

Order of Precedence (highest to lowest):

1. **env[].value** — Inline value defined directly in the Pod spec
2. **env[].valueFrom** — Single key reference (configMapKeyRef / secretKeyRef)
3. **envFrom** — Bulk import from ConfigMap or Secret

```
spec:
  containers:
  - name: app
    envFrom:
    - configMapRef:
        name: app-defaults      # LOG_LEVEL=info (lowest priority)
    env:
    - name: LOG_LEVEL
      value: "debug"            # This WINS (highest priority)
    - name: DB_HOST
      valueFrom:
        configMapKeyRef:
          name: app-overrides
          key: DATABASE_HOST    # Overrides envFrom value
```



Advanced Volume Patterns

Projected Volumes and subPath Mounts



Projected Volumes

```
spec:
  containers:
    - name: app
      volumeMounts:
        - { name: all-config, mountPath: /etc/app, readOnly: true }
  volumes:
    - name: all-config
      projected:
        sources:
          - configMap:
              name: app-config
              items: [{ key: application.yaml, path: application.yaml }]
          - secret:
              name: db-credentials
              items: [{ key: password, path: secrets/db-password }]
          - downwardAPI:
              items: [{ path: labels, fieldRef: { fieldPath: metadata.labels } }]
          - serviceAccountToken:
              { path: token, expirationSeconds: 3600, audience: vault }
```



subPath Mounts

Mount a single key without overwriting the entire directory:

❌ Without subPath

```
# This REPLACES everything
# in /etc/nginx/
volumeMounts:
- name: config
  mountPath: /etc/nginx/
  # All existing files in
  # /etc/nginx/ are hidden!
```

✅ With subPath

```
# Only adds nginx.conf,
# preserving other files
volumeMounts:
- name: config
  mountPath: /etc/nginx/nginx.conf
  subPath: nginx.conf
  # Other files in /etc/nginx/
  # remain visible!
```



Immutable ConfigMaps and Secrets

```
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config-v3 }
data:
  # ... your config keys ...
immutable: true # Cannot be changed after creation
```

✓ Benefits

- Protects against accidental changes
- Reduces API server load and improves performance
- Enables versioned config rollout



Update Strategy

- Create new ConfigMap with version suffix
- Update Deployment reference; rolling update applies it
- Delete old ConfigMap after rollout



Secret Best Practices

Security Beyond Base64



RBAC for Secrets

Restrict who can read secrets with fine-grained RBAC:

```
# Role: restrict to specific secret names
rules:
- apiGroups: ["" ]
  resources: ["secrets"]
  resourceNames: ["db-credentials", "api-keys"]
  verbs: ["get"]
---
# RoleBinding: bind ServiceAccount to Role
subjects:
- { kind: ServiceAccount, name: dns-service-sa }
roleRef:
  { kind: Role, name: secret-reader, apiGroup: rbac.authorization.k8s.io }
```

Encryption at Rest

By default, Secrets are stored unencrypted in etcd!

AWS EKS Encryption

```
# eksctl ClusterConfig snippet
secretsEncryption:
  keyARN: arn:aws:kms:...
# ...
```

Uses AWS KMS envelope encryption

What Encryption at Rest Does

- Encrypts Secret data before writing to etcd
- Decrypts transparently on read via API server
- Protects against etcd backup exposure
- Does NOT protect against API access



Enterprise Secrets with Vault

HashiCorp Vault + External Secrets Operator



Vault Architecture Overview

Core Components

- **Secrets Engines** — KV v2, database, PKI, transit
- **Auth Methods** — Kubernetes, OIDC, LDAP, AppRole
- **Policies** — path-based ACLs in HCL (deny-by-default)
- **Audit Devices** — every access logged

Why Vault for K8s?

- Centralized secret management across clusters
- Dynamic, short-lived credentials
- Full audit trail of secret access
- Versioned secrets with rollback

KV v2 is the most common engine — stores versioned key-value pairs at paths like `apps/myapp/db`

Vault + Kubernetes Auth

How Pods Authenticate to Vault

Pods use their ServiceAccount JWT token. Vault validates it against the Kubernetes API server and issues a scoped Vault token.

```
# Configure Vault Kubernetes auth
vault auth enable kubernetes
vault write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc"

# Bind a ServiceAccount to a Vault policy
vault write auth/kubernetes/role/myapp-role \
  bound_service_account_names=myapp-sa \
  bound_service_account_namespaces=production \
  policies=app-readonly \
  ttl=1h
```

Flow: Pod SA token → Vault validates with K8s API → issues Vault token with bound policies → Pod reads secrets

External Secrets Operator Pattern

ESO Components

- **SecretStore** — connection config to Vault
- **ExternalSecret** — declares what to sync
- **Operator** — reconciles on `refreshInterval`

Workloads consume standard K8s Secrets — no code changes needed.

Recommended Pattern

```
kind: ExternalSecret
spec:
  refreshInterval: 5m
  secretStoreRef:
    name: vault-store
    kind: ClusterSecretStore
  target: { name: db-credentials }
  data:
  - secretKey: username
    remoteRef: { key: apps/myapp/db, property: username }
  - secretKey: password
    remoteRef: { key: apps/myapp/db, property: password }
```



Secret Rotation Workflow

End-to-End Rotation Flow

1. **Update in Vault** — rotate credential via CLI, API, or pipeline
2. **ESO detects change** — next `refreshInterval` polls Vault
3. **K8s Secret updated** — ESO patches the native Secret
4. **Pod consumption** — volume mounts auto-update (~60s); env vars need restart

Tip: Use `stakater/Reloader` to trigger rolling restarts when Secrets change — essential for env-var consumers.

Vault + ESO Best Practices

Security

- NetworkPolicy: restrict ESO egress to Vault only
- Minimum RBAC for ESO service account
- Enable Vault audit logging
- Encrypt etcd at rest (AWS KMS)

Lifecycle

- deletionPolicy: Retain for critical secrets
- Size refreshInterval for freshness vs API cost
- Label ESO-managed Secrets for audit trail

Monitoring

- Alert on ExternalSecret non-Ready status
- Track `externalsecret_sync_calls_error` metric
- Monitor provider API rate limits



Complete Working Example

Putting It All Together

Full Example — ConfigMap + Secret

```
kind: ConfigMap
metadata: { name: web-app-config }
data:
  APP_PORT: "8080"
  nginx.conf: |
    server { listen 80; location / { proxy_pass http://localhost:8080; } }
---
kind: Secret
metadata: { name: web-app-secrets }
stringData:
  DB_PASSWORD: "s3cret!"
  API_KEY: "sk-abc123"
```

Full Example — Deployment

```
# Deployment spec.template.spec (key parts)
containers:
- name: app
  envFrom:
  - configMapRef: { name: web-app-config }
  # ... secretKeyRef for individual secrets ...
  # ... volumeMounts for files ...
volumes:
- name: nginx-config
  configMap: { name: web-app-config }
- name: secret-files
  secret: { secretName: web-app-secrets, defaultMode: 0400 }
```



Lab 5: ConfigMaps and Secrets

Hands-On Exercise

- Create ConfigMaps using literal and file approaches
- Consume ConfigMaps as environment variables and volume mounts
- Create and consume Secrets of type Opaque and TLS
- Observe hot reload behavior with volume-mounted ConfigMaps
- Use projected volumes to combine ConfigMap + Secret + Downward API
- Store secrets in HashiCorp Vault and sync via External Secrets Operator

Duration: ~30-40 minutes

Key Takeaways

1. Separate config from code — Use ConfigMaps and Secrets to make your images environment-agnostic.

2. Volume mounts > env vars — Volumes support hot reload (~60s) and are more secure for secrets.

3. Base64 ≠ encryption — Enable encryption at rest (AWS KMS) and use ESO + Vault for real workloads.

4. Use immutable resources — Prevent accidental changes and reduce API server load.

? Questions & Discussion

Module 5: Managing Application Configuration and Secrets

Up Next: Module 6 — Accessing Your Applications from Outside — Ingress & Egress